Searching in Elastic Search

1. Introduction to Searching

Concept

Searching in Elasticsearch is done using the **Search API**, where you send a query and get back:

- matching documents (hits)
- relevance scores (_score)
- **Example:**

```
GET /products/_search
{
  "query": {
    "match_all": {}
  }
}
```

✔ Returns all documents
✔ Useful for testing or debugging

2. Term-Level Queries (Exact Matching)

- **Concept**
- Do **NOT** analyze input
- Used for **exact values**
- Best for: IDs, categories, status
- Example:

```
GET /products/_search
{
  "query": {
    "term": {
      "category": "Electronics"
    }
  }
}
```

3. match query

```
"query": {
  "match": {
    "description": "smart phone"
  }
}
```

. The Match Query

- **Concept**
- Analyzes input
- Matches tokens
- Calculates score
- **Example**
- GET /products/_search

```
{
  "query": {
    "match": {
      "description": "smart phone"
    }
  }
}
```

✔ Matches documents containing:
"smart"
"phone"

4. Retrieving Documents by IDs

- **Concept**
- Direct lookup using _id (no search, no scoring)
- **Example**
- GET /products/_doc/1

- ✔ Fastest retrieval method

5. Range Queries

- **Concept**
- Search within numeric or date ranges
- **Operators:**
- gte (≥), lte (≤)
- gt, lt
- **Example**
- GET /products/_search

```
{
  "query": {
    "range": {
      "price": {
        "gte": 100,
        "lte": 500
      }
    }
  }
}
```

6. Prefix, Wildcard, and Regular Expressions

- **Prefix Query**
- "prefix": { "name": "iph" }
- ✔ Matches: iphone, iphones
- **Wildcard Query**
- "wildcard": { "name": "iph*" }
- ✔ Supports * more than characters and ? One character

7. Querying by Field Existence

- **Concept**
- Check if a field exists in documents
- ```
  {
    "query": {
      "exists": {
        "field": "price"
      }
    }
  }
  ```
- ✔ Useful for missing data filtering
  ```

8. Searching Multiple Fields

- **Concept**
- Search across multiple fields using multi_match
- ```
  {
    "query": {
      "multi_match": {
        "query": "iphone",
        "fields": ["name", "description"]
      }
    }
  }
  ```
  ```

9. Phrase Search

- **Concept**
- Matches exact phrase **in order**
- ```
  {
    "query": {
      "match_phrase": {
        "name": "iphone 13"
      }
    }
  }
  ```
- ✔ More strict than match
  ```

13. Full Text Query Types

- match
- match_phrase
- multi_match

Boolean Query

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "name": "iphone" }}
      ],
      "filter": [
        { "range": { "price": { "lte": 1000 }}}
      ],
      "must_not": [
        { "term": { "condition": "used" }}
      ]
    }
  }
}
```

Meaning:
must → required
filter → no scoring
must_not → excluded

Query vs Filter Context

Context	Scoring	Performance
Query	Yes	Slower
Filter	No	Faster

Boosting Query

- **Concept**
- Increase importance of a field
- ```
  {
    "match": {
      "name": {
        "query": "iphone",
        "boost": 2
      }
    }
  }
  ```
  ```

nested query

```
"query": {
  "nested": {
    "path": "reviews",
    "query": {
      "match": {
        "reviews.rating": 5
      }
    }
  }
}
```

Example:

```
{
  "name": "iPhone 13",
  "reviews": [
    { "user": "Ali", "rating": 5
},
    { "user": "Sara", "rating":
3 }
  ]
}
```

example

- Create index:
- { "id": 1, "name": "iPhone 13", "category": "Electronics", "price": 900, "description": "smart phone from Apple", "in_stock": true }

- { "id": 2, "name": "Samsung Galaxy S22", "category": "Electronics", "price": 800, "description": "Android smart phone", "in_stock": false }

- { "id": 3, "name": "Nike Shoes", "category": "Fashion", "price": 120, "description": "running shoes", "in_stock": true }

match (Full-text search)

```
{
  "query": {
    "match": {
      "description": "smart phone"
    }
  }
}
```

out:

- iPhone ✔
- Samsung ✔

term (Exact match)

- {
- "query": {
- "term": {
- "category": "Electronics"
- }
- }
- }
- Out:
- iPhone ✔
- Samsung ✔
- Nike ❌

Range

```
{
  "query": {
    "range": {
      "price": {
        "gte": 800,
        "lte": 900
      }
    }
  }
}
```

Bool

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "description": "smart" }}
      ],
      "filter": [
        { "range": { "price": { "lte": 900 }}}
      ],
      "must_not": [
        { "term": { "in_stock": false }}
      ]
    }
  }
}
```

summary

Query	استخدام
term	exact
match	Text(analyze text)
range	أرقام/تواريخ
bool	logic
nested	بيانات معقدة

Task

- Apply **multi_match (**fields ["name", "description"]**)**
- **Apply** prefix containing (iph)
- **Ensure the field price is exists**